

ENABLING OLFACTION FOR ROBOTICS THROUGH CONTINUAL DEEP
LEARNING AND COMPUTATIONAL CHEMISTRY

by

Kordel Kade France

APPROVED BY SUPERVISORY COMMITTEE:

Ovidiu Daescu, Chair

Shalini Prasad

Latifur Khan

Sriraam Natarajan

Yu Xiang

Copyright © 2026

Kordel Kade France

All rights reserved

*This doctoral dissertation
is dedicated to the machines
and AI who suffered
without a proper sense of olfaction
until the present time.*

ENABLING OLFACTION FOR ROBOTICS THROUGH CONTINUAL DEEP
LEARNING AND COMPUTATIONAL CHEMISTRY

by

KORDEL KADE FRANCE, BS, MS

DISSERTATION

Presented to the Faculty of
The University of Texas at Dallas
in Partial Fulfillment
of the Requirements
for the Degree of

DOCTOR OF PHILOSOPHY IN
COMPUTER SCIENCE

THE UNIVERSITY OF TEXAS AT DALLAS

May 2026

ENABLING OLFACTION FOR ROBOTICS THROUGH CONTINUAL DEEP LEARNING AND COMPUTATIONAL CHEMISTRY

Kordel Kade France, PhD
The University of Texas at Dallas, 2026

Supervising Professor: Ovidiu Daescu, Chair

Despite extraordinary progress in artificial intelligence (AI), modern systems remain incomplete representations of human cognition. Vision, audition, and language have received disproportionate attention due to well-defined benchmarks, standardized datasets, and consensus-driven scientific foundations. In contrast, *olfaction* - a high-bandwidth, evolutionarily critical sense - has been largely overlooked. This omission presents a foundational gap in the construction of truly embodied and ethically aligned super-human intelligence. This dissertation establishes that the exclusion of olfactory perception from AI architectures is not due to irrelevance but to structural challenges: unresolved scientific theories of smell, heterogeneous sensor technologies, lack of standardized olfactory datasets, absence of AI-oriented benchmarks, and inappropriate learning algorithms. Consequently, the work here makes several contributions to each of these five topics and demonstrates how adaptive and continuous machine learning methods are not only beneficial, but mandatory for advancing machine olfaction to identical standards as its visual, lingual, and auditory counterparts. I establish a design for *olfactory processing units (OPUs)* and other hardware; define new olfactory-based navigation techniques; contribute several open source and peer-reviewed datasets; and explore new temporal difference learning algorithms that enable neural networks to continuously grow, prune, and adapt their own architectures according to the complexity of the olfactory

task. Finally, I demonstrate a culmination of these topics on real applications to underscore their practicality beyond theory. I hope this dissertation inspires more work in machine olfaction and enables a new sense for embodied artificial intelligence.

PART II

ADAPTIVE AND CONTINUAL DEEP LEARNING

CHAPTER 7

CONTINUAL LEARNING AND MATRIX THEORY

Modern deep learning architectures are typically static, limiting their applicability in dynamic and non-stationary tasks. In addition, many of these modern models are based on transformers trained on petabytes of internet data. **The luxury of big data does not exist among the unstandardized and data-starved world of olfaction. As a result, I became drawn toward machine learning methods that could continuously update their own priors based on batches of observed data, and that could adjust their own architectures according to the observed data complexity. These desires led me to develop this chapter’s *Grow-Prune-Freeze (GPF)* neural network concept, built on known work in random matrix theory and scaled up from first-principles mathematics.**

GPFs introduce an adaptive framework where networks grow, prune, and freeze layers in response to performance and neuron activity. This framework is analyzed through the lens of non-linear random matrix theory (RMT), providing insight into spectral evolution, stability, and feature robustness. Specifically, I show how the work from (Pennington et al., 2017) can be extended from single-layer neural networks to n -layer continuous-learning deep neural networks that add single layers as they learn, and that eigenvalue composition is preserved. I then show that many of the RMT principles from (Martin and Mahoney, 2021) work hold as a result. The GPF framework also partially addresses both catastrophic forgetting and plasticity loss—two complementary challenges in continual learning—by combining patience-based growth, belief-hardened pruning, and stability-based neuron freezing. Theoretical analysis via the Marchenko-Pastur law and Stieltjes transforms demonstrates spectral invariance across training epochs. Experiments over a variety of domains demonstrate the extrapolation of our theory to practice. This work bridges adaptive architectures, RMT, and plasticity preservation; the principles and experimentation establish that GPFs could be a mathematically sound

offering for indefinitely scalable continual learning, but a much larger experimental battery is required to claim this absolutely. Regardless, GPFs helped me in developing many of the plume-tracking models and olfactory navigation models proposed in the succeeding chapters due to their ability to interpret the complexity of the data space and preserve temporal knowledge, and it is through these lenses that I specifically analyze GPFs mathematically.

7.1 Grow-Prune-Freeze Networks

Modern deep learning architectures are almost universally designed with fixed network topologies, where the number of layers, neurons per layer, and connectivity patterns are specified a priori. While effective in many static tasks, such designs are ill-suited for robotic applications that encounter continuous non-stationary environments, sparse sensory feedback, and dynamic task demands. Biological neural systems, by contrast, exhibit continuous structural adaptation: neurons are added, pruned, or stabilized according to activity, environmental exposure, and functional necessity. Motivated by olfactory navigation tasks in robotics, I desired a framework for adapting architectures to task complexity that mitigate many of the banes of continual learning. In other words, a neural network should start from a single layer and adapt itself to the complexity of its observations by successively adding layers and selectively pruning and freezing dead and active neurons, respectively.

Continual learning requires neural networks to adapt indefinitely to non-stationary data streams while maintaining both *stability* (retention of prior knowledge) and *plasticity* (ability to learn new tasks) (Dohare et al., 2024). Standard deep learning methods excel in the “train-once” paradigm but suffer catastrophic plasticity loss in continual settings, with performance degrading to linear baselines after thousands of tasks (Dohare et al., 2024). Concurrently, adaptive architectures that prune dynamically (Lasby et al., 2024; Frankle and Carbin, 2019) show promise but lack theoretical grounding for long-term spectral stability.

Grow-Prune-Freeze neural networks offer a framework that automatically (1) grows hidden layers when validation loss plateaus; (2) prunes “soft” neurons according to temporal belief values; and (3) freezes stable layers to prevent forgetting while preserving plasticity. As a result, I root GPFs in RMT and analogize the mathematics to theories of deep learning architectures. I empirically evaluate the framework against comparable neural networks in multiple tasks to amount support for their efficacy. In addition, I discuss how many of the principles in RMT with non-adaptive neural networks are preserved with GPFs, and how the framework can counter concerns with plasticity loss in continual learning.

Using non-linear RMT (Pennington et al., 2017), it can be shown that GPFs’ low-rank perturbations maintain Marchenko-Pastur spectral densities with shape parameter $\psi \approx 1$, ensuring dynamical isometry and gradient propagation. Specifically, I show that the work from (Pennington et al., 2017) can be extended from single-layer neural networks to n -layer continuous-learning deep neural networks that incrementally add layers as they learn while preserving eigenvalue spectrality. I then show that principles from (Tao, 2012; Martin and Mahoney, 2021) hold as a result. The final model partially addresses the three plasticity correlates identified by (Dohare et al., 2024): (1) weight magnitude growth, (2) dead units, and (3) effective rank collapse.

The contributions of GPFs are:

1. A mathematically supported framework for adaptive continuous learning;
2. RMT analysis showing spectral invariance under indefinite adaptation;
3. Tuning criterion for introduced parameters to stabilize continual learning;
4. Extension of theory to empirical validation over multiple continual learning tasks.

7.1.1 Relevant Background

Dynamic Sparse Training (DST) (Lasby et al., 2024) alternates pruning/regrowth, while MorphNet (Gordon et al., 2018) optimizes sparsity via regularization. Growing methods like

DEN (Yoon et al., 2018) add neurons on loss plateaus. General principles for constructing adaptive neural networks are established by (Mirzadeh et al., 2022) and backed by empirical support on recent model architectures.

Elastic Weight Consolidation (EWC) from Kirkpatrick, et al. (Kirkpatrick et al., 2017) penalizes important weights; Progressive Nets (Rusu et al., 2016, 2022) expand laterally. (Dohare et al., 2024) demonstrate plasticity loss across optimizers/architectures, proposing continual backpropagation (CBP) via selective reinitialization. GPFs incorporate CBP while adding structural adaptation. More work from Li, et al. (Li and Ji, 2021) and Lyle, et al. (Lyle et al., 2023) denote other challenges in building adaptive architectures that continually learn. (Pennington et al., 2017) derive spectral densities for non-linear activations, and (Cohen-Karlik et al., 2022) link low-rank bias to generalization. I extend this to *non-equilibrium ensembles* induced by adaptive perturbations through the continual adding of layers in deep models.

Cascade-Correlation networks (Fahlman and Lebiere, 1990) and Hinton’s Deep Belief Networks (Hinton et al., 2006) were early attempts at adaptive architectures, adding hidden units sequentially based on residual error. More recent approaches include Progressive Neural Networks (Rusu et al., 2016, 2017, 2022) and Grow-and-Prune methods (Fehring et al., 2025). While empirically effective, these approaches lack theoretical characterization of how structural adaptation affects network dynamics and representational capacity.

Random matrix theory (RMT) analyzes deep networks by examining spectral properties of weight and activation matrices. Pennington and Worah (Pennington et al., 2017) extended classical RMT to nonlinear activations $Y = f(WX)$, deriving limiting spectral distributions for Gram matrices $G = \frac{1}{n}Y^\top Y$. Classical RMT assumes fixed width, dense connectivity, and iid random weights - assumptions all violated in adaptive networks.

Magnitude-based pruning (LeCun et al., 1989), the *Lottery Ticket Hypothesis* (Frankle and Carbin, 2019; Chen et al., 2021), and parameter freezing (Foster et al., 2017) demonstrate

Table 7.1. GPF Network Hyperparameters

Method	Description
$k_{\max}$	Maximum training epochs
d_i	i^{th} layer dimension
ρ_e	Epoch patience required before GPF is enabled
ρ_k	Epoch patience required before growing a layer
ρ_p	Epoch patience required before pruning neurons
ρ_f	Epoch patience required before a neuron can be frozen
ω_v	Weight threshold for belief counter
ω_p	Activation threshold for pruning
ω_f	Loss threshold for neuron freezing
α	Learning rate
γ	Discount factor
ϵ	Greediness
η_g	Greediness decay
η_{bs}	Batch size decay
η_h	Belief value decay

that sparse, stabilized networks can preserve critical knowledge. In GPFs, pruning and freezing are integrated with growth to form a continuous cycle of adaptive architecture.

Olfactory robotic tasks can significantly benefit from adaptive networks due to sparse, noisy, and non-stationary sensory inputs (Crimaldi et al., 2022; Schmuker et al., 2016). GPFs allow for dynamic capacity allocation and preservation of critical features through hardened neurons, enabling plasticity and robustness.

7.2 Theoretical Analysis

The GPF framework assumes a very simple neural network to start: an input layer, an output layer, and a single hidden layer. Consider a feedforward network with L_t layers at time t , input dimension n_0 , hidden widths $n_1, \dots, n_{L_t-1}$, and output n_{L_t} . Weights $W^\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ are initialized from Kaiming distribution.

A GPF always begins with a single hidden layer $\ell_0 \in L_t$ and build up network complexity only as the data demands it. The addition of layers is dependent on several growth criteria to ensure dimensionality adaptivity to the target problem. During training, if the loss curve is decreasing but not at a global minimum, GPFs add a layer and do nothing to the previous weights because they’ve evidently learned *something*, and it is desired to preserve this eigenvalue spectrality. If the loss curve is volatile, GPFs add a layer but randomize the weights from the previous layer so spurious correlations are not learned. We thus allow the eigenvalues of the weights to “reset” for both layers.

To facilitate this, I define a threshold ω_l within the validation loss. As it learns, if the network experiences no change in loss above this threshold, a new hidden layer ℓ_i is added, where $i > 0, i \in \mathbb{R}$. There is a “patience” variable ρ_k such that there must be ρ_k epochs with validation loss all below this threshold in order to add a new layer.

The above can be denoted as follows: Add layer $\ell = L_t + 1$ if validation loss $\mathcal{L}_{\text{val}}$ satisfies:

$$\mathcal{L}_{\text{val}}^{(t-k)} - \mathcal{L}_{\text{val}}^{(t)} < \omega_l, \quad \forall i \in [t - k + 1, t], \quad (7.1)$$

where $\omega_v > 0$ is the stagnation threshold. New layer initializes $n_{L_t+1} = n_{L_t}$ neurons with $h_{i,j}^{(L_t+1)} = 0$.

Each neuron’s input weight $w_{i,j}^\ell$ has belief $h_{i,j}^\ell \in [0, 1)$. This belief value is such that if the absolute value of a neuron’s weight is consistently above a positive value ω_w (or that neuron is constantly fired) for multiple successive epochs, then it is incremented, and thus becomes “hardened”. Therefore, I perform network updates according to the following:

$$h_{i,j}^\ell(t) = h_{i,j}^\ell(t-1) + \eta_h \cdot 1 \{ |w_{i,j}^\ell(t)| > m \}, \quad (7.2)$$

where $\eta_h = 1/(k_{\text{max}})$ normalizes to approach but never reach 1 according to the current training epoch. Hardened neurons are asymptotically closer to 1 and adverse to pruning. A belief value of 1 is associated with a neuron considered as “frozen”, which I discuss shortly.

The more successive epochs each neuron lies above threshold ω_w , the more it is incremented and the closer it gets to being fully hardened and resistant to pruning. Intuitively, neurons with low belief values will be considered as soft and pruned. I consider neurons with high belief values over several epochs to have weights correlative with patterns that resemble a high belief of truth in the data. A common practice of pruning is to prune all weights below a certain threshold exemplified with LeCun’s techniques in *Optimal Brain Damage* (LeCun et al., 1989). My experiments provide evidence to suggest that not all low-magnitude weights should be pruned, and that some low-magnitude weights are prudent in order to capture patterns in sparse networks (Shazeer et al., 2017) (Frankle and Carbin, 2019) (Lasby et al., 2024). Especially after several rounds of pruning, low-magnitude weights can become useful, hence the reasoning for instantiating the separate belief value per neuron.

Parameter ω_p is a layer-wise pruning threshold. After at least ρ_b epochs have occurred since the addition of layer ℓ_i , all layers $\ell_0 - \ell_{i-1}$ prior to the one just added will be pruned such that all neurons within said layers with belief values below some threshold of ω_p will be removed from the network, or their weights set to zero.

Succinctly, prune neuron i in layers $\ell < \ell_t$ if:

$$h_{i,j}^\ell < \omega_p \quad \text{and} \quad |w_{i,j}^\ell| < d_i. \quad (7.3)$$

The hidden layer added at an epoch will never be pruned during that epoch, but may be pruned later after ρ_b “patience” epochs. For each hidden layer added, their neurons are assigned an initial belief value of $h = 0$. Each neuron must learn relationships and prove its robustness to sufficiently correlative patterns within the data in order to achieve relevance for the network. This process of growing the network through hidden layers and shrinking it through pruning should occur indefinitely, but may be set by the practitioner to slowly decay over time if desired.

Another set of tunable parameters are ρ_f and ω_f , where $\rho_f < k_{\text{max}}$ and $\omega_f \in [0, 1]$.

I freeze layer ℓ_i if, for a patience of ρ_f epochs, $100.0 * \omega_f\%$ weights satisfy:

$$|w_{i,j}^\ell(k_{\max} - q) - w_{i,j}^\ell(k_{\max})| < \omega_f, \quad \forall q \in [k_{\max} - \rho_f + 1, k_{\max}]. \quad (7.4)$$

Layers are frozen to deter catastrophic forgetting and preserve plasticity, and borrow many of the principles from Dohare, et al. in (Dohare et al., 2024). Freezing layers preserves knowledge of key patterns learned earlier in training while also ensuring dependencies needed in subsequent layers by layer ℓ_n for pattern recognition are preserved.

Batch size scheduling according to data heteroscedasticity, network size, and state of convergence has been shown useful in many prior efforts (Adachi et al., 2024; Devarakonda et al., 2018; Smith and Topin, 2018). I also found value with batch scheduling in GPF networks. As a result, one may set a batch size decay η_{bs} according to loss criterion such that a given batch size b_t is initialized according to the following: $b_t = b_0 \cdot \exp(-\eta_{bs} \mathcal{L}_{\text{val}}^{(t)})$.

As I will soon show, freezing stable layers also ensures that the spectral distribution of the Gram matrix are preserved for proving the asymptotic eigenvalue density.

7.2.1 Some Notes on Training

As the network begins training, it does not know how many layers it will eventually add. I design the network to grow, prune, and freeze its parameters in perpetuity such that the model is always adapting to the data. This is a strong deviation from today's large language models that are over-parameterized but generally perform well for their target benchmarks. I desire sparse models that grow in depth but necessarily and sufficiently parameterize themselves to the data they perceive. In this manner, the concepts of training and testing being two separate processes are foregone. For GPF networks, feedback is directly incorporated according to its experience or training data and there is no separate test phase. I design GPF networks to be employed in scenarios where there is too little data for conventional training mechanisms and/or where the data may contain temporal patterns that change over time according to new

dynamics introduced into the environment, as is the case with plume tracking in olfactory robotics.

7.2.2 Analysis via Random Matrix Theory

Pennington, et al. (Pennington et al., 2017) showed that the distribution of eigenvalues from the data covariance matrix is idempotent after passing through a single nonlinear layer of a neural network. I aim to show that this principle extrapolates to any number of layers $L \in [1, \infty]$ as long as (1) stable depth-wise growth occurs within the network according to (Javed et al., 2023), (2) pruning is done according to the Hessian approximation in (LeCun et al., 1989), and (3) layers are frozen at the appropriate time to preserve pruned knowledge. I show how principles from (Schoenholz et al., 2016) govern the limit for how deep a network can be given a set of hyperparameters, and how decisions on growth can occur based on their assumptions.

To maintain consistency with the current state of the art, I set up my notation in the spirit of Pennington, et al. (Pennington et al., 2017). Let $X \in \mathbb{R}^{n_0 \times m}$ be a data matrix (with m data samples), and let $W \in \mathbb{R}^{n_1 \times n_0}$ be a random weight matrix with i.i.d. entries, e.g.

$$W_{ij} \sim \mathcal{N}(0, \sigma_w^2/n_0).$$

Define the pre-activation matrix

$$Z = WX,$$

and apply a pointwise nonlinearity $f : \mathbb{R} \rightarrow \mathbb{R}$ to obtain the post-activation matrix

$$Y = f(Z),$$

where f is applied elementwise.

In random matrix theory, W and X are both random weights and input data, respectively, for easy experimentation. Schoenholz, et al. (Schoenholz et al., 2016) showed how the affects

of learning can be analyzed through the flow of information through random networks. In a real deep learning problem, these matrices and networks may *appear* random, but may be initialized with techniques such as Kaiming initialization. I preserve this notion to align with conventional random matrix theory, but later show experimentally how the principles in studying random matrices translate to explanatory data from which patterns exist to be learned. Here, Y is a primitive vehicle for understanding more complex deep learning cases that extrapolate from basic RMT principles.

One can define the empirical Gram matrix as:

$$M = \frac{1}{m} Y^\top Y \in \mathbb{R}^{m \times m}.$$

The main object of interest is the *limiting spectral distribution (LSD)* of M as $n_0, n_1, m \rightarrow \infty$ at comparable rates (e.g. such that their ratios stay constant). By applying the classical *moments method* from random matrix theory, one can derive an equation for the trace of the quotient

$$R(z) = (M - zI)^{-1},$$

and thereby characterize the asymptotic eigenvalue density of M . I carry the formula for R into a different form articulated shortly.

I extend the above to support multiple layers (weight matrices) dynamically added according to specified criteria. Each layer of a neural network can be interpreted as a repeated - initially random - projection. Now consider a network with L layers, with weight matrices $W^{(1)}, W^{(2)}, \dots, W^{(L)}$, data matrix X , and repeated application of linear transforms and nonlinearity. At each layer ℓ , one may define

$$Z^{(\ell)} = W^{(\ell)} Y^{(\ell-1)}, \quad Y^{(\ell)} = f(Z^{(\ell)}),$$

starting from $Y^{(0)} = X$.

One can now ascertain under what conditions an RMT analysis similar to the single-layer case can extend to the Gram (or covariance) matrix of the multi-layer $Y^{(L)}$. In particular, we can derive the limiting spectral distribution of such Gram matrix:

$$M^{(L)} = \frac{1}{m} [Y^{(L)}]^\top Y^{(L)}$$

With the multilayer Gram matrix, we can now reformulates the multi-layer spectral limit so the depth L is the principal parameter. I present the Gram recursion as an operator composition and give conditions that guarantee stable limits as layers are added. With $X^{(0)} \in \mathbb{R}^{n_0 \times m}$ as the input data matrix, and for each layer $\ell = 1, \dots, L$ I set

$$Z^{(\ell)} = W^{(\ell)} X^{(\ell-1)},$$

with $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ random weight matrices satisfying the following assumptions extended from (Pennington et al., 2017) (Martin and Mahoney, 2021):

1. **Random weights.** Independently across layers, each weight matrix satisfies the following at its initialization:

$$(W_{ij}^{(\ell)}) \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}\left(0, \frac{\sigma_\ell^2}{n_{\ell-1}}\right)$$

2. **All layers are initialized with identical width.** All layers $\ell_i \in L$ are initialized with d width. Layer width never exceeds d but can be pruned to be lower than d .
3. **Nonlinearity.** The activation f is measurable, of at most polynomial growth, and satisfies

$$\mathbb{E}[f(g)^2] < \infty, \quad \mathbb{E}[g f(g)] < \infty \quad \text{for } g \sim \mathcal{N}(0, 1).$$

These conditions ensure that the moments integrals appearing below are finite. As stated in Section 7.3, my experiments only address ReLU-related activations.

4. **Input data.** The columns of $X^{(0)}$ have bounded Euclidean norm, and the empirical Gram matrix

$$M^{(0)} = \frac{1}{m} (X^{(0)})^\top X^{(0)}$$

converges in distribution (e.g. in the sense of moments) to a nonrandom limit.

With these assumptions satisfied, one can now define the layer- ℓ Gram matrix as:

$$M^{(\ell)} = \frac{1}{m} [X^{(\ell)}]^\top X^{(\ell)}. \quad (7.5)$$

and its equivalent Stieltjes transform by

$$s^{(\ell)}(z) = \frac{1}{m} \text{Tr}((M^{(\ell)} - zI)^{-1}). \quad (7.6)$$

The nonlinearity of activation functions precludes many principles from random matrix theory applying to neural networks. While many of the tools for assessing moments and spectral densities from matrix theory still apply, I show how these tools can be further employed to analyze space folds with ReLU activations (and friends) (Lewandowski et al., 2025).

Let Ψ_ℓ be an activation function that acts as a *nonlinear spectral transformer*, encoding how a random layer reshapes the spectral statistics of the preceding layer. Explicit expressions for Ψ_ℓ depend on integrals involving the activation $\psi \in \Psi$ and may reduce to polynomial or rational forms for common nonlinearities. Under the moments-method derivation (counting non-linear random walks at each layer) one obtains, for each ℓ , a deterministic operator Ψ_ℓ that enables Equation 7.6 to be written as

$$s^{(\ell)}(z) = \Psi_\ell(s^{(\ell-1)}(z), z)$$

for every $z \in \mathbb{C} \setminus \mathbb{R}$, where Ψ_ℓ depends on the activation ψ , layer variance σ_ℓ^2 . Thus, when depth is increased by adding layers, the transform is updated by *composition* of these operators. We can define the depth- L composition operator

$$\Psi^{(L:1)} = \Psi_L \circ \Psi_{L-1} \circ \cdots \circ \Psi_1,$$

so that the Stieltjes transform at depth L is succinctly written as

$$s^{(L)}(z) = \Psi^{(L:1)}(s^{(0)}(z), z). \quad (7.7)$$

7.2.3 Homogeneous Layers

A key clarification in the above methodology is the distinction between networks with homogeneous and heterogenous hidden layers. A network with homogeneous hidden layers is one whose layers are all the same size and shape - each hidden layer has the same number of neurons, activation functions, and other properties as its preceding and successive hidden layer. I demonstrate the above methods over such networks and leave the case for when the network adds in a heterogeneous layer (the less common case) as an extension for future work. I note that, due to the pruning process of GPFs, the network is bound to have heterogeneous layers. This is intended and addressed by the rationale above. What is explicitly *not* addressed with GPFs is adding a layer with a different number of neurons than the previous layer. As I describe “homogeneous layers” moving forward, I do so in the sense that GPFs only add layers whose initial non-pruned dimensionality is identical to that of the original non-pruned hidden layer.

Proceeding under this assumption, suppose each layer is identically distributed with the same aspect ratios and variances, so $\psi_\ell \equiv \Psi$ for all $\ell \in Z$. Then by taking Equation [7.7](#) above, we have:

$$s^{(L)}(z) = \underbrace{\psi \circ \psi \circ \dots \circ \psi}_{L \text{ times}}(s^{(0)}(z), z) = \Psi^{\circ L}(s^{(0)}(z), z).$$

Two immediate consequences follow:

1. **Monotone / contractive iteration.** If $\psi(\cdot, z)$ is a contraction on a suitable subset $\mathcal{S}$ of analytic transforms (for example, relative to the supremum norm on compact

z -domains), then the sequence $s^{(L)}(z)$ converges geometrically in L towards the unique fixed point $s^*(z)$ solving $s^* = \Psi(s^*, z)$.

2. **Operator accumulation.** Even when Ψ is not strictly a contraction globally, one may obtain convergence for initial conditions $s^{(0)}(z)$ in a basin of attraction; numerically this corresponds to the spectral measure “forgetting” low-rank or sample-specific components as depth increases. I note, however, that this “forgetting” is partially countered by the layer-specific freezing process defined above.

7.2.4 Fixed-Point Depth-Limit Theorem

I attempt to formalize the conditions for convergence of GPFs at any layer depth limit with the following theorem:

Theorem 1 (Depth-Limit under Composition). *Suppose the layers are homogeneous so that $\psi_\ell = \Psi \forall \psi \in \Psi$. Assume there exists an open set $\mathcal{U} \subset \mathbb{C} \setminus \mathbb{R}$ containing the spectral support of $M^{(\ell)}$ for all ℓ , and a Banach space $(\mathcal{S}, \|\cdot\|)$ of analytic transforms on $\mathcal{U}$ such that:*

1. *Condition 1: $\Psi : \mathcal{S} \times \mathcal{U} \rightarrow \mathcal{S}$ is well-defined and analytic in its first argument for each fixed $z \in \mathcal{U}$.*
2. *Condition 2: For each fixed $z \in \mathcal{U}$, the Fréchet derivative $D_s \Psi(s, z)$ exists and there is a constant $\rho(z) < 1$ with*

$$\sup_{s \in \mathcal{S}} \|D_s \Psi(s, z)\| \leq \rho(z) < 1.$$

Then for every initial Stieltjes transform $s^{(0)} \in \mathcal{S}$, the iterates $\{s^{(L)}\}_{L \geq 0}$ defined by $s^{(L)} = \Psi^{\circ L}(s^{(0)}, \cdot)$ converge in $\mathcal{S}$ to the unique fixed point $s^(\cdot)$ that satisfies $s^*(z) = \Psi(s^*(z), z)$ for all $z \in \mathcal{U}$. Consequently, the empirical spectral distribution of $M^{(L)}$ converges (as $m, n_0, \dots, n_L \rightarrow \infty$, jointly with $L \rightarrow \infty$) to the deterministic law given by the Stieltjes transform s^* .*

Remarks.

- Condition 2 is a sufficient contraction-style condition; weaker conditions (e.g., spectral radius < 1 only on a subset or average contractivity) can also be used in applications and are often verifiable by bounding the nonlinear moments that define $D_s\Psi$.
- For inhomogeneous layers one obtains $s^{(L)} = \Psi_L \circ \dots \circ \Psi_1(s^{(0)})$ and conclusions about convergence require joint control of the products of derivatives $\prod_{\ell=1}^L D_s\Psi_\ell$. A sufficient criterion is that there exist ρ_ℓ with $\sup_\ell \rho_\ell < 1$ and $\|D_s\Psi_\ell\| \leq \rho_\ell$, which yields geometric decay of perturbations. However, I leave the formal analysis of nonhomogeneous layers for evaluation outside of this dissertation.
- From a combinatorial (moments) perspective, adding depth extends the class of contributing walks by allowing arbitrarily many layer-to-layer transitions; the operator Ψ summarizes the net effect of summing those additional contributions at each depth increment.

7.2.5 Pseudocode Overview

In consolidation of the above, I can succinctly define how the GPF network evolves according to the following rules:

1. **Layer Growth:** Add a new hidden layer if validation loss stagnates over ρ_v epochs:

$$\Delta L_{\text{val}}^{(t-k:t)} < \omega_v$$

2. **Neuron Belief Hardening:** Each neuron i has belief $h_i \in [0, 1)$, incremented if

$$|w_i| > \omega_v \text{ over } k_{\text{max}} \text{ epochs.}$$

3. **Pruning and Layer Freezing:** Neurons with $|w_i| < \omega_v$ and $h_i < \omega_p$ are pruned. Layers with $> 100.0 \times \omega_f\%$ stable weights over ρ_f epochs are frozen.

Below is the pseudocode quantifying the above rules into a manner that can be executed for the experiments in the following sections.

```
 $\Omega \leftarrow$  initialize network with single hidden layer
randomize weights of  $\Omega$  via Glorot init
initialize all h-values of  $\Omega$  to zero
for each epoch t or until termination criteria:
    train  $\Omega$  on batch b
    update  $\Omega$  weights and h-values
    if t > patience:
        if validation loss stagnates for k epochs:
            add new hidden layer
        prune neurons with low h and small weights
        freeze stable layers satisfying criteria v
        update h-values
```

7.3 ReLU Activations and Spectral Density

Many modern deep learning architectures make use of the rectified linear unit (ReLU) activation function, or some derivative thereof. The performance garnered through the implementation of ReLU is well known and I intentionally constrain the use of activation functions within hidden layers strictly to ReLU and ReLU variants (e.g. GeLU). The reasoning for this is two-fold: constraining the hidden layers to one activation function allows me implement a form of control while ablating other architectural parameters; secondly, the transforms over the data that occur due to the non-linearities imposed by ReLU offer several points of interesting analysis with respect to GPFs.

When the layers are ReLU or bounded activations, these integrals are often explicit or numerically tractable, enabling concrete verification of the above stability conditions. The

work of Lewandowski, et al. (Lewandowski et al., 2025) note how ReLU networks transform input space in a piecewise-linear manner, producing folded, non-convex geometries across layers. Each fold corresponds to a region where the activation pattern changes, and consecutive layers compound these folds, increasing expressivity while also affecting the distribution of activations and gradients.

In the context of GPFs, adding a new hidden layer introduces additional ReLU folds, effectively increasing the number of non-linear regions in the network. This operation changes the activation covariance matrix and the Jacobian, which are central objects in RMT. Consequently, the Gram matrix ($G^{(l)} * t = \frac{1}{n * l - 1} (Y_t^{(l)})^\top Y_t^{(l)}$) experiences shifts in its spectrum, potentially increasing the spectral width and introducing new spikes corresponding to strongly activated neurons.

GPF neurons that become hardened due to high belief values (h_i) tend to preserve specific folds over successive epochs. In spectral terms, this generates persistent low-rank structures in the activation covariance, aligning with the “spiked” eigenvalues studied in RMT. These spikes can be interpreted as geometrically meaningful folds that represent robust features in the input space.

Pruning neurons with low activity or low belief selectively removes folds that contribute little to the effective expressivity of the network. Spectrally, this reduces bulk eigenvalues associated with inactive directions, sharpening the separation between critical spikes and the background, thus enhancing signal-to-noise ratios in the network’s representation.

Finally, freezing layers preserves existing folds while allowing newly added layers to introduce additional complexity. From an RMT perspective, freezing stabilizes the spectral distribution of weights of previously learned features, preventing catastrophic drift of critical eigenvalues while maintaining plasticity in subsequent layers.

Adaptive architectures like GPFs can dynamically allocate representational capacity to active plume regions while pruning inactive neurons. Hardened neurons preserve critical

correlations, and frozen layers prevent catastrophic forgetting. I hypothesize that this mechanism improves efficiency, plasticity, and robustness in sensory-limited, dynamic environments such as olfactory robotics, which we mentioned was our primary motivation for the GPF design. Growing the network accounts for plume complexity, pruning it adaptively filters out spurious time-correlated events, and freezing layers captures time-dependent patterns at different parts of the plume.

7.4 Width vs Depth

Research from Mirzadeh, et al. (Mirzadeh et al., 2022) define a key set of architectural boosters for continual learning that I found worked well in practice. Namely, limiting the use of global average pooling and replacing it with max pooling and batch normalization seemed to limit plasticity loss as they claim. Where I digress, however, is the argument that wider and shallower networks promote more stable continual learning, as I did not see this in my experiments. Nearly every experiment I ran showed that deeper networks (albeit sparse after pruning) outperformed the shorter, wider networks. My final model from (Paul et al., 2025) discussed in Chapter 6 was obviously over-parameterized at 16 layers, but the depth aided in modeling the prototype distribution from such a small sample size.

The work from Mirzadeh, et al. is evidently correct, and I think the difference in experimental observations can be drawn from our training data size. In most of my experiments, I am modeling with a small amount of training data which seems to benefit from the added depth. If my applications had more plentiful training data, I expect that my results would converge toward their's and we would reach the same conclusions regarding the benefits of width.

I note this as a point for future research beyond the doctoral studies.

7.5 Experiments

To demonstrate the *grow-prune-freeze* methodology, I selected three different tasks with many known benchmarks in literature: image classification over CIFAR10, reinforcement learning over the Atari environment, and text generation via GPT-2.

For each task, I trained a baseline model and a GPF variant of the same model. Both models were trained under identical parameters with only one exception: the baseline model begins training at n hidden layers while the GPF model begins training with one hidden layer and grows to a maximum depth of n layers. All hyperparameters, data, training procedures, and seeds are identical otherwise. Architectural details for each model can be observed in Section I of the Appendix.

7.5.1 GPF Task: Image Classification

I selected image classification over CIFAR10 because it provides a foundational entry point and because it allows us to build off the assumptions and experiments from the similar work by Dohare, et al. (Dohare et al., 2024). In reference to Figure 7.1, one will notice that the validation loss begins to diverge for the baseline model, but steadily decreases for the dynamic GPF model. This may indicate that GPFs can provide more stable learning even while simultaneously adding layers and pruning neurons.

Figure 7.2 shows the weight eigenvalues after each is added. In observation of the eigenvalue spectra over several different CIFAR batches, the empirical spectral density (ESD) of the eigenvalues indeed follow the Marchenko-Pastur distribution - a characteristic that Pennington, et al. and Martin, et al. indicate provides evidence of stable weight eigenvalues during learning (Pennington et al., 2017) (Martin and Mahoney, 2021).

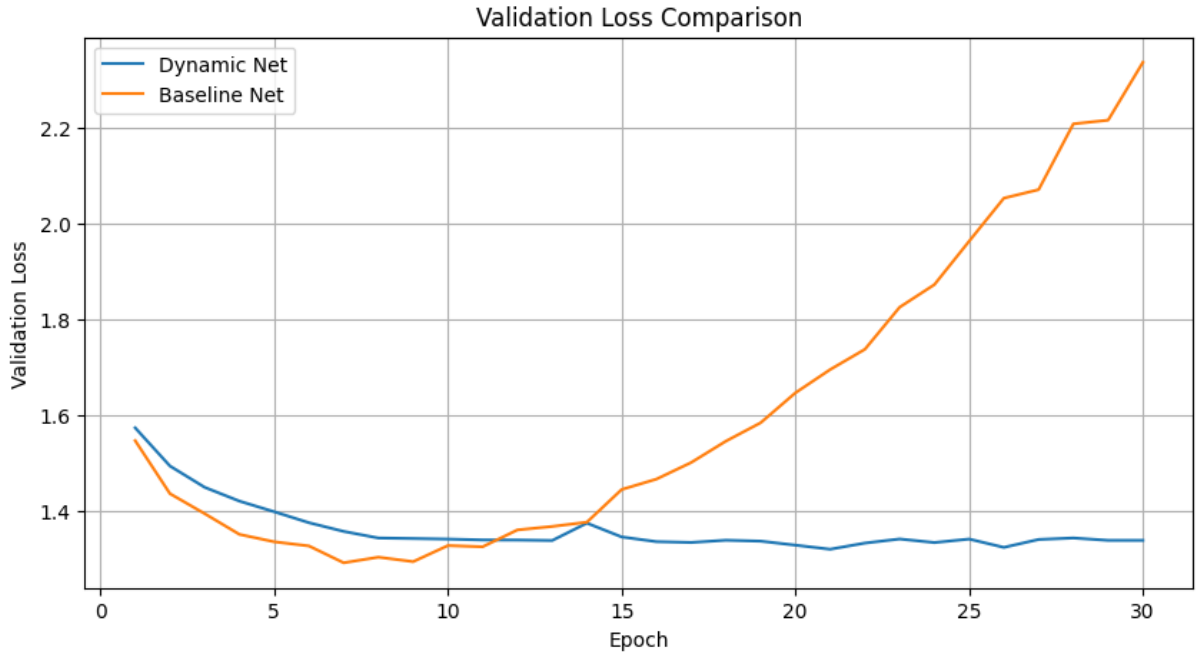


Figure 7.1. Validation losses of a GPF network and a baseline static neural network trained under identical parameters for the CIFAR image classification task.

7.5.2 GPF Task: Reinforcement Learning

Reinforcement learning contains many tasks and benchmarks from which one could select, but I evaluated over the Atari environment in order to assess GPF’s decision making capabilities. Specifically, I selected the *Ms Pacman* environment as it was also leveraged for the experiments in Chapter 8. Conclusions are difficult to draw from the accompanying figure in 7.3, but one can immediately observe that the volatility in validation rewards for the baseline policy is much higher than that of the dynamic GPF policy.

A more comprehensive evaluation over the entire Atari suite is an intuitive segue for future work.

7.5.3 GPF Task: GPT-2

Finally, given today’s attraction toward and the many recent advances in large language models, I select GPT-2 as a means to show how GPFs can be extended to generative

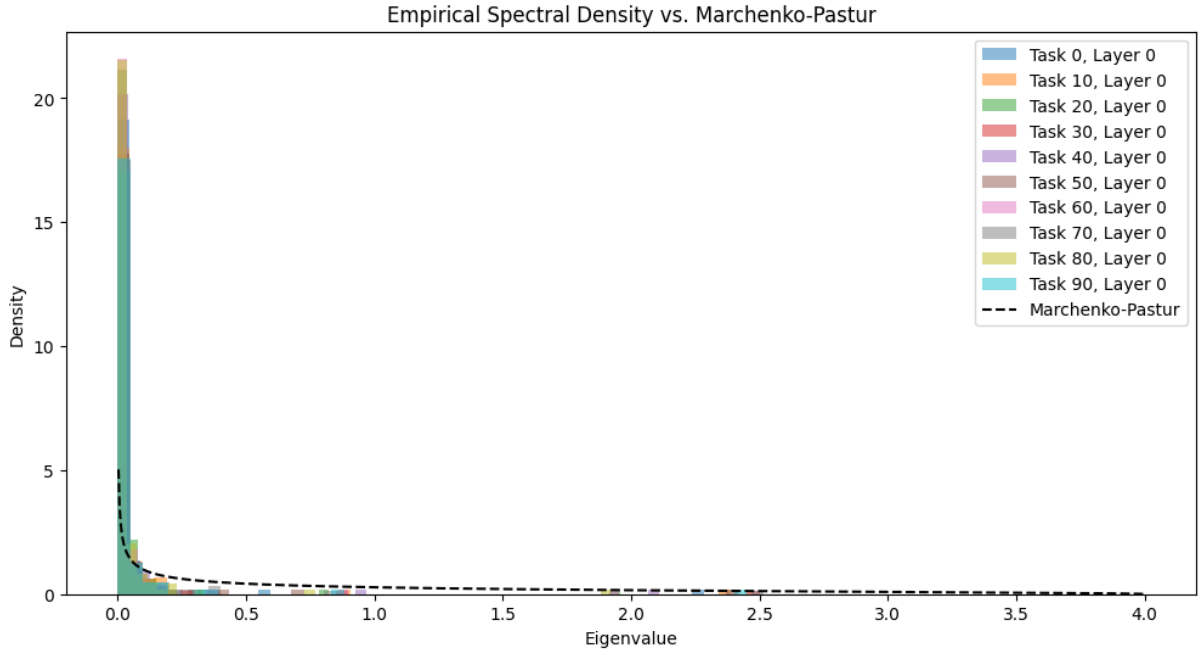


Figure 7.2. The eigenvalue spectra for the weights of the GPF network from the CIFAR image recognition task follow the Marchenko-Pastur distribution, maintaining the hypothesis of (Pennington et al., 2017).

tasks and much larger architectures. I acknowledge that GPT-2 is no longer considered “state-of-the-art”, but the thorough evaluation of GPFs over the much larger state-of-the-art architectures is outside of the financial and computational budget for a single PhD student. The architectures of many of the notable large language models today were derived from GPT-2 (Sebastian Raschka, Sebastian Raschka; Olmo et al., 2025; OpenAI et al., 2025), so I intuit that using it as a vehicle for analysis is a representative architecture of what might occur in large model.

I use the *TinyShakespeare* dataset from Karpathy (Karpathy, 2015) to train a baseline model and GPF variant of GPT-2. Figure 7.4 shows the final loss plot for the baseline GPT-2 model. Interestingly, one can see that convergence is not achieved and the loss is very volatile. Conversely, the training loss for the GPF variant of GPT-2 in Figure 7.5 shows good convergence, with a steady decay after epoch 2000. GPF GPT-2 achieved the maximum layer

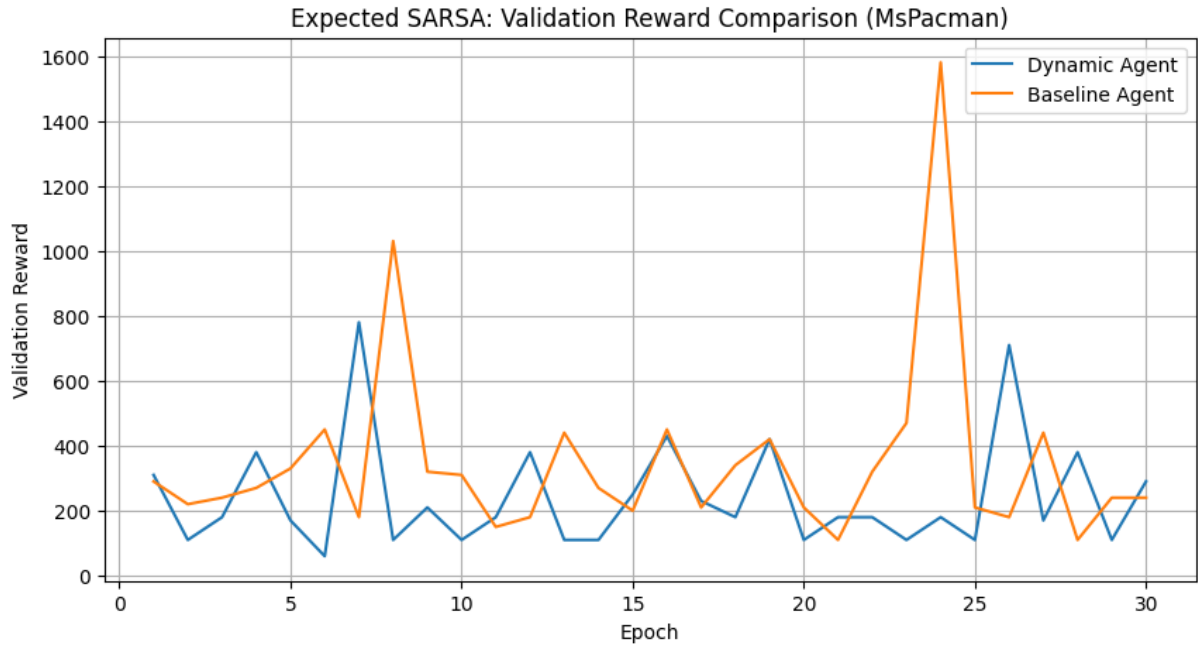


Figure 7.3. Validation losses of a GPF policy and a baseline static policy trained under identical parameters for the Atari task. Both policies are neural networks based on Expected SARSA.

count at around epoch 2500 and ceased growing for the rest of training. Consistent with the work of LeCun (LeCun et al., 1989) and Frankle, et al. (Frankle and Carbin, 2019), the percentage of neurons pruned decays as training progresses. For GPFs specifically, this may indicate that neurons from earlier weights have hardened their belief states around values that reflect true patterns in the data.

My intent with the tasks above was to show initial evidence that the principles of the GPF framework can generalize to different models and tasks. There is much more to investigate in relation to the Pareto frontiers of the conventional “train-and-freeze” paradigm — just as humans continuously learn and update priors as they gather more experiences, so can we enable AI models to do the same. However, I acknowledge that my work here is only evidence supporting the GPF framework for continual learning, and that a thorough evaluation over a large battery of tasks and different architecture types is prudent future work.

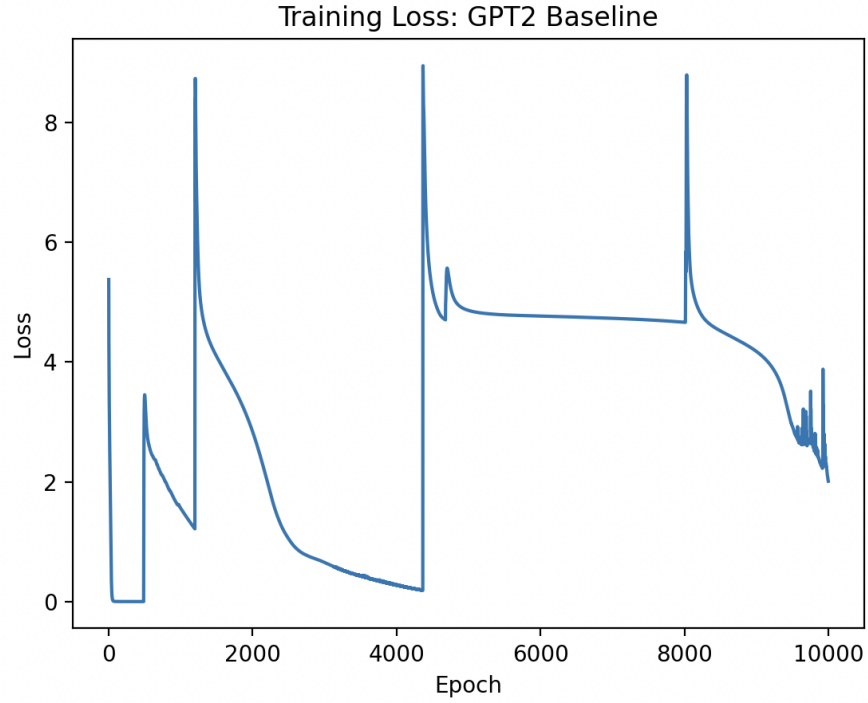


Figure 7.4. Training loss of the GPT-2 baseline. Training parameters were identical to the GPF variant below.

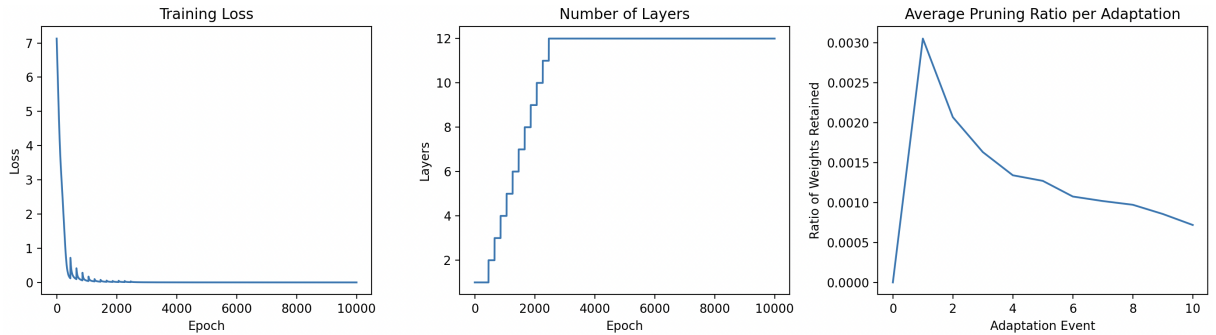


Figure 7.5. Training details for the GPF variant of GPT-2. (Left) The validation loss shows steady convergence over training, even while growing and pruning its own parameters. (Center) The GPF reached the maximum number of layers at around 2500 epochs and kept training. (Right) The pruning ratio over each adaptation event shows that the number of weights pruned is much higher at the beginning of training, but tapers as eigenvalues stabilize.

7.6 GPFs: A Work in Progress

Grow-Prune-Freeze networks were my attempt to satisfy multiple frustrations in building machine learning models that could continually learn in a controlled manner and with little data. In the absence of plentiful data to provide statistical grounding in olfactory tasks, I sought a framework that was mathematically grounded, and RMT proved to be a solid foundation from which I could build more principles. By keeping the architecture of GPFs simple - with homogeneous layers and constrained activation functions - I was able to conduct a more controlled battery of ablative experiments to quantify their performance. The architectural simplicity also allows for recursive calls of the same hidden layer - since all layers are identical, a machine learning model need only define one hidden layer, and can just load the i^{th} indexed weights for the i^{th} recursion call. Computational complexity remains the same, but size complexity may be noticeably smaller in very deep architectures that conventionally export layers sequentially. This becomes most prudent on edge devices and robots that need to make decisions with onboard compute. The above optimization for a single recursively-called hidden layer is another requirement derived from Chapter 11's olfactory navigation models which focus on edge-based compute.

GPFs are still a work in progress, and while I have made honest attempts to mathematically and empirically justify their worth in context of other surveyed research, I do not claim that their development is final. Based on early experiments, GPFs show competitive performance with their static counterparts and with less computational demand. There is still much more work to be done to calibrate and optimize their design, but what has been detailed thus far proved useful for informing olfactory model design in the chapters that follow.